Chapter 19: Interceptors

1. Overview

Every HTTP request an Angular application sends and every response it receives can be intercepted, inspected, and rewritten before it reaches its destination. That is the job of an **interceptor**: a piece of code that sits between `HttpClient` and the actual `XMLHttpRequest` / `fetch` call, forming a **chain of responsibility** through which every request flows out and every response flows back.

Interceptors are the idiomatic place to implement cross-cutting HTTP concerns without polluting every service that calls `HttpClient`:

- Attaching an `Authorization` header / auth token to outgoing requests.
- Centralized error handling (401 → redirect to login, 5xx → toast notification).
- Retry logic with backoff for transient network failures.
- Logging / analytics / request timing.
- Request/response transformation (e.g., unwrapping an envelope, converting date strings to `Date` objects, adding a `Content-Type` header).
- Showing/hiding a global loading spinner.
- Caching GET responses.

Angular has **two interceptor APIs**:

	Legacy (class-based)	**Modern (functional)**
Introduced	Angular 4.3 (`HttpClientModule`)	Angular 15 (`provideHttpClient`), stable/recommended since Angular 16-17
Shape	Class implementing `HttpInterceptor`	Plain function matching `HttpInterceptorFn`
Registration	`HTTP_INTERCEPTORS` multi-provider token	`provideHttpClient(withInterceptors([...]))`
DI	Constructor injection	`inject()` inside the function
Boilerplate	Higher (class, `@Injectable`, `intercept()` method)	Lower (a single function)
Standalone-app friendly	Requires `NgModule`-style provider array or `importProvidersFrom`	First-class citizen of the standalone/bootstrapApplication world
Status (Angular 17+)	Still supported, not deprecated, but not the recommended default for new code	Recommended default

Since Angular 15, `HttpClient` itself can be provided via `provideHttpClient()` instead of importing `HttpClientModule`, and this is the entry point through which functional interceptors are wired in.

2. Core Concepts

2.1 The `HttpInterceptorFn` signature (functional, modern)

```
type HttpInterceptorFn = (
  req: HttpRequest<unknown>,
  next: HttpHandlerFn
) => Observable<HttpEvent<unknown>>;
```

- `req` — the outgoing, **immutable** `HttpRequest`.
- `next` — a function you call with a (possibly modified) request to forward it to the next interceptor in the chain (or to the backend if this is the last one).
- Return value — an `Observable<HttpEvent<unknown>>`, which is exactly what `next(req)` itself returns, so most interceptors simply `return next(req.clone({...}))` possibly piped through RxJS operators.

A functional interceptor is *just a function* — no class, no `@Injectable()`, no decorator. It is registered in an array, and Angular calls it with the right arguments; DI values are obtained via `inject()` since the function runs inside an injection context.

2.2 The `HttpInterceptor` interface (class-based, legacy)

```
interface HttpInterceptor {
  intercept(
    req: HttpRequest<any>,
    next: HttpHandler
  ): Observable<HttpEvent<any>>;
}
```

- `next` is an object (`HttpHandler`) with a `.handle(req)` method rather than a callable function.
- Must be `@Injectable()` and registered via the multi-provider token `HTTP_INTERCEPTORS`.
- Dependencies come through the constructor, the normal Angular DI way.

```
@Injectable()
export class AuthInterceptor implements HttpInterceptor {
  constructor(private auth: AuthService) {}

  intercept(req: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
    const cloned = req.clone({
      setHeaders: { Authorization: `Bearer ${this.auth.token}` }
    });
    return next.handle(cloned);
  }
}
```

Registered as:

```
providers: [
  { provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor, multi: true }
]
```

The `multi: true` is mandatory — it tells Angular's DI this token resolves to an **array** of providers rather than a single overriding value; forgetting it silently replaces all previously registered interceptors with only the last one provided.

2.3 Registering functional interceptors

```
// app.config.ts
export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(
      withInterceptors([authInterceptor, errorInterceptor, loggingInterceptor])
    )
  ]
};
```

- The array order **is** the execution order for the outgoing leg (see Internal Working, §4).
- `withInterceptors()` only accepts `HttpInterceptorFn`s. To mix in legacy class-based interceptors registered via `HTTP_INTERCEPTORS`, add the companion function `withInterceptorsFromDi()`:

```
provideHttpClient(
  withInterceptors([authInterceptor]),
  withInterceptorsFromDi() // pulls in HTTP_INTERCEPTORS-registered classes too
)
```

Without `withInterceptorsFromDi()`, class-based interceptors registered the old way are **silently ignored** when the app uses `provideHttpClient()` instead of `HttpClientModule` — a very common migration gotcha.

2.4 Immutability of `HttpRequest`/`HttpResponse`

`HttpRequest` and `HttpResponse` instances are immutable. You never mutate `req.headers` or `req.url` directly (most properties are `readonly`, and `HttpHeaders`/`HttpParams` are immutable value objects). Instead you call `.clone({...})`, which returns a *new* request object with the specified overrides while copying everything else. This matters because:

- The same request object might be read by multiple interceptors, retried, or logged; mutation would cause spooky action at a distance.
- `req.clone()` supports both `setHeaders`/`setParams` (merge semantics) and `headers`/`params` (full replace semantics) — mixing these up is a common bug source.

2.5 Request vs. response phase

An interceptor conceptually has two phases:

1. **Request (outbound) phase** — code that runs before calling `next(...)`. Used for cloning/mutating the request (adding headers, changing URL, adding params).

2. **Response (inbound) phase** — code that runs on the Observable returned by `next(...)`, typically via RxJS operators (`tap`, `map`, `catchError`, `retry`, `finalize`). Used for transforming/logging responses or handling errors.

```
export const timingInterceptor: HttpInterceptorFn = (req, next) => {
  const started = Date.now(); // outbound
  return next(req).pipe(
    tap(event => {                // inbound
      if (event.type === HttpEventType.Response) {
        console.log(`${req.url} took ${Date.now() - started}ms`);
      }
    })
  );
};
```

2.6 `HttpEvent` stream, not a single value

`next(req)` does not emit just the final response — by default `HttpClient` requests emit a single `HttpResponse`, but if `reportProgress: true` is set, the Observable emits a whole sequence of `HttpEvent` types: `HttpSentEvent`, `HttpHeaderResponse`, `HttpUploadProgressEvent`, `HttpDownloadProgressEvent`, finally `HttpResponse`. An interceptor that blindly assumes "one response" and uses `map` on the raw event stream can crash on progress events; always narrow with `event.type === HttpEventType.Response` (or use `filter`) before treating the event as a response body.

2.7 Multiple interceptors — composition & order

Interceptors compose like middleware / Russian dolls: request order is chain-array order, response order is the reverse.

```
[A, B, C] registered  →  outgoing:  A → B → C → backend
                         incoming:  backend → C → B → A
```

This symmetry (LIFO on the way back) is fundamental — it means an interceptor that mutates the request in the outbound phase naturally gets first look at the raw response in the inbound phase, mirroring a stack unwind.

2.8 Interceptors and DI scope

- Functional interceptors registered app-wide via `provideHttpClient(withInterceptors([...]))` in `app.config.ts` apply to **every** `HttpClient` injected in the app by default.
- `HttpClient` can also be scoped: if you provide `provideHttpClient(...)` again inside a specific route's providers or a lazy-loaded feature's providers, that creates a **new instance** of `HttpClient` (and its own handler chain) confined to that injector subtree — useful for a micro-frontend module needing a different base URL/auth scheme without affecting the rest of the app.
- Interceptors run once per request per `HttpClient` instance's chain, not once globally.

2.9 What interceptors do *not* touch

- `fetch()` calls, `XMLHttpRequest` used directly, third-party HTTP libraries (axios), WebSocket connections, and `<img src>`/browser-native resource loads all bypass Angular's interceptor chain entirely, because the chain is built *inside* `HttpClient`. Only requests issued through Angular's `HttpClient` are intercepted.
- (Since Angular 17, there is an experimental `withFetch()` provider that switches `HttpClient`'s backend from `XMLHttpRequest` to `fetch` — interceptors still apply the same way in that case, since interception happens above the backend abstraction.)

3. Code Examples

3.1 Auth token interceptor (functional)

```
import { HttpInterceptorFn, HttpErrorResponse } from '@angular/common/http';
import { inject } from '@angular/core';
import { catchError, throwError } from 'rxjs';
import { AuthService } from './auth.service';
import { Router } from '@angular/router';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService);
  const router = inject(Router);

  // Don't attach the token to auth/login endpoints to avoid loops.
  const isAuthUrl = req.url.includes('/auth/login') || req.url.includes('/auth/refresh');
  const token = auth.getAccessToken();

  const authReq = (!isAuthUrl && token)
    ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
    : req;

  return next(authReq).pipe(
    catchError((err: HttpErrorResponse) => {
      if (err.status === 401 && !isAuthUrl) {
        auth.clearSession();
        router.navigate(['/login']);
      }
      return throwError(() => err);
    })
  );
};
```

3.2 Error handling + retry with exponential backoff (functional)

```
import { HttpInterceptorFn, HttpErrorResponse } from '@angular/common/http';
import { inject } from '@angular/core';
import { throwError, timer } from 'rxjs';
import { retry, catchError } from 'rxjs/operators';
import { NotificationService } from './notification.service';
```

```
const MAX_RETRIES = 3;
const isRetryable = (status: number) =>
  status === 0 || status === 502 || status === 503 || status === 504;

export const errorRetryInterceptor: HttpInterceptorFn = (req, next) => {
  const notifier = inject(NotificationService);

  // Never retry non-idempotent verbs blindly.
  const idempotent = req.method === 'GET' || req.method === 'HEAD';

  return next(req).pipe(
    idempotent
      ? retry({
          count: MAX_RETRIES,
          delay: (error: HttpErrorResponse, retryCount) => {
            if (!isRetryable(error.status)) {
              // Non-retryable error: rethrow immediately, skip remaining retries.
              return throwError(() => error);
            }
            const backoffMs = Math.min(1000 * 2 ** (retryCount - 1), 8000);
            const jitter = Math.random() * 250;
            return timer(backoffMs + jitter);
          }
        })
      : (source: typeof req extends never ? never : any) => source, // no-op for non-
idempotent
    catchError((err: HttpErrorResponse) => {
      const message = err.error?.message ?? err.statusText ?? 'Unexpected error';
      notifier.showError(`Request failed: ${message}`);
      return throwError(() => err);
    })
  );
};
```

A cleaner version that avoids the awkward conditional operator by branching explicitly:

```
export const errorRetryInterceptor: HttpInterceptorFn = (req, next) => {
  const notifier = inject(NotificationService);
  const isIdempotent = req.method === 'GET' || req.method === 'HEAD';

  const response$ = next(req);

  const withRetry = isIdempotent
    ? response$.pipe(
        retry({
          count: MAX_RETRIES,
          delay: (error: HttpErrorResponse, retryCount) => {
            if (!isRetryable(error.status)) return throwError(() => error);
            const backoffMs = Math.min(1000 * 2 ** (retryCount - 1), 8000);
            return timer(backoffMs + Math.random() * 250);
          }
        })
```

```
      )
    : response$;

  return withRetry.pipe(
    catchError((err: HttpErrorResponse) => {
      notifier.showError(`Request failed: ${err.error?.message ?? err.statusText}`);
      return throwError(() => err);
    })
  );
};
```

3.3 Logging interceptor (functional, request + response timing)

```
import { HttpInterceptorFn, HttpEventType, HttpResponse } from '@angular/common/http';
import { tap, finalize } from 'rxjs/operators';

export const loggingInterceptor: HttpInterceptorFn = (req, next) => {
  const start = performance.now();
  console.groupCollapsed(`[HTTP] ${req.method} ${req.urlWithParams}`);
  console.log('Request headers:', req.headers.keys().map(k => `${k}:
${req.headers.get(k)}`));
  if (req.body) console.log('Request body:', req.body);

  return next(req).pipe(
    tap(event => {
      if (event.type === HttpEventType.Response) {
        const res = event as HttpResponse<unknown>;
        console.log(`Response [${res.status}]:`, res.body);
      }
    }),
    finalize(() => {
      console.log(`Elapsed: ${(performance.now() - start).toFixed(1)}ms`);
      console.groupEnd();
    })
  );
};
```

3.4 Response transformation interceptor (unwrap an envelope)

```
// Backend returns { data: T, meta: {...} }; interceptor unwraps to T.
import { HttpInterceptorFn, HttpEventType, HttpResponse } from '@angular/common/http';
import { map } from 'rxjs/operators';

export const unwrapEnvelopeInterceptor: HttpInterceptorFn = (req, next) => {
  return next(req).pipe(
    map(event => {
      if (event.type === HttpEventType.Response && event.body && typeof event.body ===
'object' && 'data' in (event.body as any)) {
        return event.clone({ body: (event.body as any).data });
      }
      return event;
```

```
    })
  );
};
```

3.5 Composition & order — full registration example

```
// app.config.ts
import { ApplicationConfig } from '@angular/core';
import { provideHttpClient, withInterceptors, withInterceptorsFromDi, withFetch } from
'@angular/common/http';
import { authInterceptor } from './interceptors/auth.interceptor';
import { errorRetryInterceptor } from './interceptors/error-retry.interceptor';
import { loggingInterceptor } from './interceptors/logging.interceptor';
import { unwrapEnvelopeInterceptor } from './interceptors/unwrap-envelope.interceptor';

export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(
      // Order matters: this is the OUTGOING order.
      withInterceptors([
        loggingInterceptor,        // 1st out: logs the raw outgoing request
        authInterceptor,           // 2nd out: attaches token, handles 401
        errorRetryInterceptor,     // 3rd out: retries transient failures
        unwrapEnvelopeInterceptor  // 4th out (closest to backend): unwraps body
      ]),
      withInterceptorsFromDi() // still honor legacy HTTP_INTERCEPTORS if any exist
    )
  ]
};
```

Incoming (response) order is the mirror image: `unwrapEnvelopeInterceptor` sees the raw response first (so it unwraps before anyone else sees the envelope), then `errorRetryInterceptor` (so it can retry based on the real HTTP error/body), then `authInterceptor` (catches 401 after retries are exhausted), then `loggingInterceptor` (logs the final, fully processed response last).

This ordering is deliberate: retry should generally sit *closer to the backend* than the auth interceptor if you want a fresh token attached on each retried attempt to actually happen before the retry (in this simplified example the token isn't refreshed per retry — a production refresh-token interceptor would combine auth + retry logic together, see §5).

4. Internal Working — How the Chain Is Built

4.1 The handler chain as nested functions

Conceptually, `HttpClient.request()` does not iterate interceptors imperatively at call time; it **builds a chain of nested handler functions once**, where each interceptor wraps the next handler. This is the classic decorator/middleware pattern, similar to Express or Redux middleware.

Given interceptors `[A, B, C]` and a terminal `backendHandler`, Angular effectively constructs:

```
handler = (req) => A(req, (req2) => B(req2, (req3) => C(req3, (req4) =>
backendHandler(req4))))
```

So calling `handler(req)`:

1. `A` runs its outbound logic, then calls its `next` — which is really "B wrapped with C wrapped with backend".
2. That inner call runs `B`'s outbound logic, then calls *its* `next` — "C wrapped with backend".
3. `C` runs, calls the real `backendHandler`, which performs the actual XHR/fetch call and returns an `Observable<HttpEvent>`.
4. The Observable returned from `backendHandler` propagates back up: `C`'s `.pipe()` operators run on it first, then whatever `B` piped onto the Observable `C` returned, then `A`'s.

This is precisely why the **response phase is LIFO relative to the request phase** — it is a direct consequence of function nesting/closures, not a separately maintained "response chain".

4.2 `HttpHandler` vs. bare functions

For legacy class-based interceptors, Angular represents each link as an `HttpHandler` object (`{ handle(req): Observable<HttpEvent> }`), and `HttpInterceptingHandler` iterates the `HTTP_INTERCEPTORS` array building a linked list of handler objects at injection time, each delegating to the next.

For functional interceptors, Angular's newer implementation (`HttpInterceptorHandler` internally, via `chainedInterceptorFn`) reduces the interceptor array into a single composed function using something conceptually equivalent to:

```
function chainInterceptors(interceptors: HttpInterceptorFn[], backend: HttpHandlerFn):
HttpHandlerFn {
  return interceptors.reduceRight(
    (next, interceptor) => (req) => interceptor(req, next),
    backend
  );
}
```

`reduceRight` is the key: it folds from the *last* interceptor toward the *first*, so the first interceptor in the array ends up as the outermost wrapper — the first to see the request and the last to see the response. This matches the desired declared-order-is-request-order semantics.

4.3 Where `HttpClient.request()` sits

`HttpClient.request(...)` doesn't call the backend directly — it calls `this.handler.handle(req)` where `handler` is whatever chain was constructed from the configured interceptors + the true `HttpBackend` (which wraps `XMLHttpRequest` or, with `withFetch()`, the Fetch API). This is why interceptors are invisible to code calling `HttpClient` — from the caller's perspective, `http.get(url)` still just returns an `Observable<T>`; all the chain machinery is internal.

4.4 Execution timing — lazy, per-subscription

Like all RxJS-based APIs, nothing actually executes until something subscribes to the Observable returned by `http.get()` / `http.post()` /etc. This means:

- If you call `http.get(url)` but never `.subscribe()` (or pipe into `async`), **no interceptor runs and no request is sent** — a classic beginner bug ("I called the service but nothing happened").
- Each `.subscribe()` on the same unsubscribed/cold Observable re-runs the *entire chain* including all interceptor logic — relevant when interceptors have side effects (e.g., logging fires per subscription, not per Observable creation).

5. Edge Cases & Gotchas

Order dependency. Interceptor order is a semantic decision, not just a preference. Concretely:

- An auth interceptor should generally run **before** logging so the log doesn't miss the attached token (or after, if you don't want tokens ever logged) — decide deliberately.
- If a caching interceptor sits before an auth interceptor in the outbound direction, a cache hit could bypass the auth interceptor's request cloning entirely, serving a cached response without ever re-validating auth state — this is usually fine (cache = short-circuit), but it means the caching interceptor must itself decide whether the outbound phase even calls `next()`.
- Retry logic combined with token refresh has a subtle ordering trap: if the auth interceptor is "outside" the retry interceptor, a retried request goes out with the *same, possibly now-stale* token as the original attempt, because the auth interceptor already ran once and won't re-run on the retry (retry re-invokes `next()` at the point in the chain where `retry()` was applied, not from the top of the whole chain). To refresh the token on 401 before retry, the refresh logic typically needs to live in (or be coordinated with) the same interceptor that performs the retry, or use a shared "refresh in flight" subject that all requests await.

Cloning immutably. `req.clone({ headers: newHeaders })` **replaces** the headers object entirely, while `req.clone({ setHeaders: {...} })` merges. Using `headers:` when you meant to merge silently drops every header set by an earlier interceptor (e.g., wiping out a `Content-Type` set upstream). The same distinction applies to `params` / `setParams`. Always prefer `setHeaders` / `setParams` inside interceptors unless you deliberately want to overwrite everything.

Never mutate in place. `req.headers.set(...)` returns a new `HttpHeaders` instance and does not mutate the original — forgetting to use the return value (`req.headers.set('X', 'Y'); return next(req);` without reassigning) is a no-op bug that's easy to miss because it doesn't throw.

Infinite retry loops. `retry()` without a `count` / `delay` policy set retries **forever** on any error, including a 404 or a malformed request that will never succeed — this can silently hang the app or hammer the backend. Always bound retries with `count`, and only retry genuinely transient failures (network drop, `status === 0`, 502/503/504) — never retry 4xx client errors, and never retry non-idempotent verbs (`POST` / `PATCH` / `DELETE`) unless the backend guarantees idempotency (e.g., via an idempotency key), or you risk duplicate side effects (double-charging a payment, duplicate resource creation).

Auth-refresh loops. An interceptor that, on a 401, calls a refresh-token endpoint and retries the original request must exclude the refresh endpoint itself from triggering another refresh — otherwise a truly expired refresh token causes infinite recursive 401 → refresh → 401 → refresh calls. Guard with a URL check (as in §3.1) and/or a "refreshing" flag/ `Subject` that queues concurrent requests instead of firing parallel refresh

calls.

Interceptors don't apply to non-`HttpClient` traffic. `fetch()`, raw `XMLHttpRequest`, third-party HTTP clients (axios, apollo-client's own fetch link unless explicitly bridged), image/font/script tag loads, WebSocket/SSE connections, and Service-Worker-level fetches are entirely outside Angular's interceptor chain. Teams migrating legacy code that used `fetch` directly are often surprised that their new auth interceptor "isn't firing" for some calls — the fix is routing all HTTP traffic through `HttpClient`.

Class interceptors silently dropped after migrating to `provideHttpClient()`. If a codebase still registers interceptors via `{ provide: HTTP_INTERCEPTORS, useClass: ..., multi: true }` but the app was migrated to standalone bootstrap with `provideHttpClient(withInterceptors([...]))` and nobody added `withInterceptorsFromDi()`, those legacy interceptors stop running with no error — a common silent regression during Angular version upgrades.

Order of `withInterceptors` vs `withInterceptorsFromDi`. Functional interceptors from `withInterceptors()` always run **before** DI-based (class) interceptors pulled in via `withInterceptorsFromDi()`, regardless of where `withInterceptorsFromDi()` appears in the `provideHttpClient()` argument list — this is a fixed internal ordering, not configurable by argument position.

`multi: true` omission (legacy). Forgetting `multi: true` on an `HTTP_INTERCEPTORS` provider doesn't error — it just makes that provider **replace** the entire array (or be replaced by a later single-provider registration), so other interceptors mysteriously stop running.

Per-request opt-out. Sometimes you need a specific request to skip an interceptor (e.g., skip auth header for a public asset). The standard pattern is a custom `HttpContextToken`:

```
export const SKIP_AUTH = new HttpContext().set(new HttpContextToken(() => false), true);
// usage: http.get(url, { context: new HttpContext().set(SKIP_AUTH_TOKEN, true) })
```//
and checking `req.context.get(SKIP_AUTH_TOKEN)` inside the interceptor — URL string matching (`req.url.includes(...)`) works but is brittle; `HttpContext` is the type-safe, refactor-proof mechanism intended for exactly this.

Interceptors run per `HttpClient` instance. If a lazy-loaded feature module/route provides its own `provideHttpClient(...)`, it gets its own handler chain — app-level interceptors registered only at the root do **not** automatically apply there unless also configured for that injector, which surprises teams doing micro-frontend-style module isolation.

Testing gotcha. `HttpClientTestingModule`/`provideHttpClientTesting()` intercepts at the `HttpBackend` level (via `HttpTestingController`), so interceptors registered through `provideHttpClient(withInterceptors([...]))` **do** still run in tests (unlike some naive assumptions) — but if a test only provides `HttpTestingModule` without also providing the interceptor array, no interceptor logic executes, which can mask bugs that only manifest with the token attached.

6. Interview Questions & Answers

Q1. What is an HTTP interceptor in Angular, and why would you use one?

A: An interceptor is a function (or class) that sits in the request/response pipeline between `HttpClient` and the network backend, letting you inspect or transform outgoing requests and incoming responses. Common uses: attaching auth tokens, centralized error handling, retry logic, logging, request/response shape transformation, and caching — all without modifying every individual service call.

Q2. What's the difference between the functional `HttpInterceptorFn` and the legacy class-based `HttpInterceptor`?
Interviewer intent: checks whether the candidate is current with Angular 15+ APIs, not just the pre-2023 mental model.
A: `HttpInterceptorFn` is a plain function `(req, next) => Observable<HttpEvent>` registered via `provideHttpClient(withInterceptors([...]))`, using `inject()` for DI, and it's the modern default for standalone apps. `HttpInterceptor` is a class implementing `intercept(req, next: HttpHandler)`, registered as a multi-provider on `HTTP_INTERCEPTORS`, using constructor injection. Functionally they achieve the same thing; the functional form has less boilerplate and fits the standalone-app model better. Both can coexist if you add `withInterceptorsFromDi()` to `provideHttpClient()`.

Q3. If you register interceptors as `[A, B, C]` via `withInterceptors()`, what's the execution order for the request going out, and for the response coming back?
A: Outgoing request order: A → B → C → backend. Incoming response order is the mirror/LIFO: backend → C → B → A. This falls out of how the chain is built — each interceptor wraps the next as a nested function, so the first interceptor is the outermost wrapper, seeing the request first and the response last.

Q4. Why must you call `req.clone()` instead of mutating the request object directly?
A: `HttpRequest` (and `HttpHeaders`/`HttpParams`) are immutable by design — most properties are `readonly`. Mutating them isn't actually possible for most fields, and even for the mutable-looking ones (like calling `.set()` on headers), the methods return new instances rather than mutating in place. Immutability prevents one interceptor's edits from corrupting the shared request object as it travels through the rest of the chain, avoids surprises on retried/replayed requests, and keeps the pipeline's behavior predictable and testable.

Q5. What's the difference between `req.clone({ headers: h })` and `req.clone({ setHeaders: {...} })`?
A: `headers:` performs a full replace — the request's header collection becomes exactly `h`, discarding anything set by earlier interceptors. `setHeaders:` merges the given key/value pairs into the existing headers, preserving everything else. The same distinction exists for `params` vs `setParams`. Using `headers:`/`params:` inside a mid-chain interceptor is a common bug that silently erases upstream interceptors' work.

Q6. How would you implement retry-with-exponential-backoff for transient network failures, and what must you guard against?
Interviewer intent: tests whether the candidate just knows `retry()` exists or actually understands its failure modes in production.
A: Use RxJS `retry({ count, delay })` where the `delay` callback computes `baseDelay * 2^(attempt-1)` (optionally with jitter) and returns a `timer(ms)` Observable, and inside `delay` you inspect the error and rethrow immediately (`throwError`) for non-retryable errors (4xx, non-idempotent methods) instead of retrying. Guards: bound the retry count (unbounded `retry()` retries forever, even on permanent failures like 404), only retry idempotent methods (GET/HEAD, or PUT if truly idempotent) to avoid duplicating side effects on POST, and only retry genuinely transient statuses (0, 502, 503, 504) rather than all errors.

Q7. How does a caching interceptor typically avoid calling the real backend?
A: In the outbound phase, it checks an in-memory (or storage-backed) cache keyed by URL+params; if there's a hit, it returns `of(cachedResponse)` immediately instead of calling `next(req)` — short-circuiting the chain entirely for that request. If there's a miss, it calls `next(req)`, and in the inbound phase (via `tap`) stores the resulting `HttpResponse` in the cache before passing it through.

Q8. Why doesn't an interceptor fire when you call `fetch()` directly in a component?
A: Interceptors are wired into `HttpClient`'s internal handler chain; they only run for requests dispatched through `HttpClient.get/post/put/delete/request(...)`. Anything bypassing `HttpClient` — raw `fetch`, `XMLHttpRequest`, third-party HTTP libraries, WebSocket connections, or native resource loads like `<img src>` — never enters that chain, so no interceptor logic executes for it.

Q9. You migrated an app from `HttpClientModule` to `provideHttpClient()`, and existing class-based interceptors stopped running. Why, and how do you fix it?
A: `provideHttpClient()` alone only wires up functional interceptors passed via `withInterceptors()`. It doesn't automatically pull in interceptors registered the legacy way via `{ provide: HTTP_INTERCEPTORS, useClass: X, multi: true }`. You need to add `withInterceptorsFromDi()` as an additional feature to `provideHttpClient(...)` so DI-registered class interceptors are included in the chain alongside functional ones.

Q10. How would you skip an interceptor (e.g., the auth interceptor) for a specific request, like a public health-check endpoint?
A: The robust approach is `HttpContext`: define an `HttpContextToken<boolean>` (e.g., `SKIP_AUTH`), pass it via `http.get(url, { context: new HttpContext().set(SKIP_AUTH, true) })`, and inside the interceptor check `req.context.get(SKIP_AUTH)` before attaching the token. This is type-safe and doesn't rely on brittle URL string matching, which breaks if the URL changes.

Q11. Two interceptors both need to know about the auth token — one attaches it, another retries requests. What ordering problem can arise, and how do you handle a 401 that occurs because the token expired mid-session?
Interviewer intent: probes for real production experience with token refresh, not just textbook retry knowledge.
A: If retry is "inside" (closer to backend than) auth, a retried request replays with the exact same (now possibly stale) token, since the auth interceptor already ran once for that request and won't re-run per retry attempt — retry re-invokes `next()` from where `retry()` sits in the chain, not from the top. The typical fix is to have the interceptor that catches 401 trigger a token refresh call, hold/queue any other in-flight requests (e.g. via a shared `BehaviorSubject`/`ReplaySubject` flag), clone the *original* request with the fresh token once refresh completes, and re-issue it explicitly — rather than relying on a generic `retry()` operator to magically get a new token.

Q12. Are interceptors global across the whole app, or can they be scoped?
A: By default, interceptors registered in `provideHttpClient(withInterceptors([...]))` at the root apply to every `HttpClient` instance created from that injector, i.e., app-wide. However, if a lazy-loaded route or feature module provides its own `provideHttpClient(...)` in a child injector, that creates a separate `HttpClient` instance with its own chain — root-level interceptors don't automatically apply unless also configured there. This lets different parts of an app (e.g., a micro-frontend calling a different API with different auth) use distinct interceptor sets.

Q13. What actually gets built when you register `[A, B, C]` as interceptors — is it a loop that runs three functions in sequence, or something else?
A: It's not a loop over the array at request time; it's a composed chain of nested functions built (conceptually) via `reduceRight`: `chain = A.bind(null, B.bind(null, C.bind(null, backend)))`-style nesting, so the final `handler` is a single function where calling it invokes `A`, whose `next` argument is itself a function wrapping `B` wrapping `C` wrapping the real backend handler. This closures-based structure is exactly why the response phase naturally unwinds in reverse order — it's a consequence of function composition, not a separate mechanism.

Q14. Why should you avoid putting a `console.log` of the full request/response in a logging interceptor that ships to production, and how would you address it?
A: Logging the full request can leak sensitive data — auth tokens, PII in request/response bodies, session identifiers — into browser consoles or, if piped to a remote log service, third-party log stores. Fix: gate the interceptor behind an environment flag (`environment.production ? noop : verbose`), redact known-sensitive headers/fields (`Authorization`, `password`, `ssn`) before logging, and prefer structured logging services with proper access control over raw `console.log` in shipped code.

7. Quick Revision Cheat Sheet

- **Functional interceptor**: `(req, next) => Observable<HttpEvent>`, registered via `provideHttpClient(withInterceptors([...]))`, uses `inject()`. Recommended default since Angular 15+.
- **Class interceptor**: implements `HttpInterceptor.intercept(req, next: HttpHandler)`, registered via `{ provide: HTTP_INTERCEPTORS, useClass, multi: true }`. Needs `multi: true` or it silently replaces other interceptors.
- Mixing both: add `withInterceptorsFromDi()` to `provideHttpClient()`, or class interceptors are silently dropped.
- **Order**: array order = outbound order; response order is the exact reverse (LIFO), a direct result of nested function composition.
- Always `req.clone({ setHeaders / setParams })` to merge; `headers:`/`params:` fully replaces and can wipe earlier interceptors' work. Never mutate in place — `HttpRequest`/`HttpHeaders`/`HttpParams` are immutable.
- Retry: bound with `count`, only retry idempotent methods + transient statuses (0/502/503/504), add backoff + jitter, rethrow immediately on non-retryable errors inside the `delay` callback.
- Auth-refresh-on-401 needs custom coordination (queue/flag + explicit reissue with new token), not a bare `retry()`.
- Interceptors only intercept `HttpClient` traffic — not `fetch`, raw `XMLHttpRequest`, WebSockets, or `<img>`/native resource loads.
- `HttpContext`/`HttpContextToken` is the type-safe way to opt specific requests in/out of an interceptor's behavior (e.g., skip auth for public endpoints).
- Interceptors are scoped to the `HttpClient` instance/injector that provided them — a child injector's own `provideHttpClient()` gets its own separate chain.
- Nothing runs until `.subscribe()` — interceptor logic (including side effects like logging) executes once per subscription, not once per call.
- Watch `HttpEventType` — `next(req)` can emit progress events, not just the final response; narrow with `event.type === HttpEventType.Response` before treating the body as final.

Created By - Durgesh Singh